

레몬헬스케어 AI 영업 코치

MVP — Single Context Sales System

버전	v1.0 MVP
작성일	2026년 3월
대상	Kes 단독 사용 → 팀 확장
목적	딜 관리 + AI 코치 + 음성 메모 + Win 공식

이 문서로 개발자가 바로 시작할 수 있어야 한다

- 추가 질문 없이 코딩 시작 가능한 수준으로 모든 스펙을 정의한다
- 기술 스택, DB 스키마, API 엔드포인트, 화면 구성, 구현 순서를 모두 포함한다
- AI 프롬프트 설계와 음성 업로드 파이프라인을 상세히 기술한다

1. 프로젝트 개요

영업 담당자(Kes)가 고객과 나누는 대화를 AI가 함께 정리하고, 다음에 무엇을 해야 이길 수 있는지 가이드하는 도구. 쌓인 딜 데이터에서 Win 패턴을 자동 추출하여 영업 전략을 고도화한다.

1.1 핵심 가치

- 영업은 메모(텍스트 or 음성)만 입력한다 → AI가 나머지를 정리한다
- Win/Loss 결과가 쌓일수록 공식이 정교해진다
- 어느 단계(발굴~계약)에서든 딜을 시작할 수 있다
- 음성 녹음 파일 업로드 → 자동 텍스트 변환 → AI 구조화

1.2 MVP 범위 (Phase 1 — Kes 단독)

포함	딜 CRUD, 미팅 메모(텍스트+음성), AI 코치, 제안서 초안, Win 공식(EMR·규모별)
제외	팀 멀티유저, 견적서 PDF, 계약서 DOCX, MIS 연동 (Phase 2 이후)
사용자	Kes 1명 — Supabase Auth 이메일 로그인
배포 환경	Vercel (HTTPS, 외부 접근 가능 URL)

2. 기술 스택

Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS
Database	Supabase (PostgreSQL 15) + Supabase Storage (음성 파일)
Auth	Supabase Auth — 이메일/패스워드 (단일 계정)
Backend Logic	Supabase Edge Functions (Deno) — AI 호출 서버사이드 처리
AI — 텍스트	Anthropic Claude Sonnet 4.6 API
AI — 음성→텍스트	OpenAI Whisper API (whisper-1 모델)
문서 생성	제안서: 마크다운 텍스트 렌더링 (MVP), PDF/DOCX는 Phase 2
배포	Vercel Pro (Next.js 네이티브, 자동 HTTPS)
이메일 알림	Resend API (팔로업 기한 알림)
패키지 관리	npm / pnpm

2.1 환경 변수 목록

```
# .env.local
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ... # Edge Function 전용

ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-... # Whisper STT용

RESEND_API_KEY=re... # 이메일 알림
RESEND_FROM_EMAIL=noreply@yourdomain.com
```

3. 프로젝트 디렉토리 구조

```
lemon-sales-coach/
├── app/                                # Next.js App Router
│   ├── (auth)/
│   │   └── login/page.tsx            # 로그인 화면
│   ├── (dashboard)/
│   │   ├── layout.tsx                # 인증 가드 + 사이드바
│   │   ├── page.tsx                 # 홈 대시보드
│   │   ├── deals/
│   │   │   ├── page.tsx              # 딜 목록
│   │   │   ├── new/page.tsx          # 딜 생성
│   │   │   │   └── [id]/
│   │   │   │       ├── page.tsx      # 딜 상세 + AI 코치
│   │   │   │       └── proposal/page.tsx # 제안서 초안
│   │   └── win-formula/page.tsx      # Win 공식 대시보드
│   └── api/
│       ├── analyze-memo/route.ts     # 메모 → Claude 구조화
│       ├── transcribe/route.ts       # 음성 → Whisper → 텍스트
│       ├── coach/route.ts            # AI 코치 응답
│       ├── proposal/route.ts         # 제안서 초안 생성
│       └── win-patterns/route.ts     # Win 패턴 집계
├── components/
│   ├── deals/
│   │   ├── DealCard.tsx
│   │   ├── DealKanban.tsx
│   │   ├── DealForm.tsx
│   │   └── StageProgress.tsx
│   ├── memo/
│   │   ├── MemoInput.tsx            # 텍스트 입력
│   │   └── AudioUpload.tsx          # 음성 업로드
│   ├── ai/
│   │   ├── CoachPanel.tsx           # AI 코치 결과
│   │   └── ContextSnapshot.tsx      # 고객 프로파일 카드
│   └── ui/                           # 공통 컴포넌트
├── lib/
│   ├── supabase/
│   │   ├── client.ts                # 브라우저용 클라이언트
│   │   └── server.ts                 # 서버용 클라이언트
│   ├── ai/
│   │   ├── claude.ts                # Claude API 래퍼
│   │   ├── whisper.ts               # Whisper API 래퍼
│   │   ├── prompts.ts               # 프롬프트 템플릿 모음
│   │   └── utils.ts
│   ├── types/
│   │   └── index.ts                 # Deal, MeetingLog 등 타입 정의
│   └── supabase/
│       ├── migrations/              # DB 마이그레이션 SQL
│       └── functions/
│           └── process-audio/        # 음성 처리 Edge Function
```

4. 데이터베이스 스키마

모든 테이블에 Row Level Security (RLS) 적용. `user_id = auth.uid()` 조건으로 Kes 본인 데이터만 접근.

4.1 deals — 딜 마스터

컬럼명	타입	제약	설명
<code>id</code>	<code>uuid</code>	PK, default <code>gen_random_uuid()</code>	딜 고유 ID
<code>user_id</code>	<code>uuid</code>	FK → <code>auth.users</code> , NOT NULL	담당자 (Kes)
<code>hospital_name</code>	<code>text</code>	NOT NULL	병원명
<code>services</code>	<code>text[]</code>	NOT NULL	서비스 목록 (레몬케어 등)
<code>stage</code>	<code>text</code>	NOT NULL, default 'discovery'	발굴/제안/협상/확정/계약
<code>prob_grade</code>	<code>text</code>	nullable	확도 S/A/B/C
<code>entry_point</code>	<code>text</code>	NOT NULL, default 'discovery'	딜 시작 단계
<code>hospital_size</code>	<code>text</code>	nullable	의원/병원/종합병원/상급종합
<code>emr_vendor</code>	<code>text</code>	nullable	유비케어/비트/이지케어텍 등
<code>decision_makers</code>	<code>jsonb</code>	default '[]'	[{name, role, style, contact_pref}]
<code>context_snapshot</code>	<code>jsonb</code>	default '{}'	AI 추출 맥락 전체 (핵심 필드)
<code>stage_log</code>	<code>jsonb</code>	default '[]'	[{stage, entered_at, exited_at, days}]
<code>prob_history</code>	<code>jsonb</code>	default '[]'	[{date, from, to, trigger}]
<code>outcome</code>	<code>text</code>	nullable	win / loss / active
<code>win_factors</code>	<code>text[]</code>	nullable	계약 성사 요인
<code>loss_reason</code>	<code>text</code>	nullable	드랍 원인 분류
<code>days_in_stage</code>	<code>integer</code>	default 0	현재 단계 체류일 (자동계산)
<code>next_action</code>	<code>text</code>	nullable	다음 액션
<code>due_date</code>	<code>timestamptz</code>	nullable	팔로업 기한
<code>created_at</code>	<code>timestamptz</code>	default now()	생성일
<code>updated_at</code>	<code>timestamptz</code>	default now()	최종 수정일

`context_snapshot` 구조: `{ hidden_needs[], pain_points[], objections[], win_signals[], decision_maker_style, contact_preference, budget_sensitivity, decision_log[{date, note}] }`

4.2 meeting_logs — 미팅 기록

컬럼명	타입	제약	설명
<code>id</code>	<code>uuid</code>	PK	로그 고유 ID
<code>deal_id</code>	<code>uuid</code>	FK → <code>deals</code>	연결된 딜
<code>user_id</code>	<code>uuid</code>	FK → <code>auth.users</code>	작성자
<code>input_type</code>	<code>text</code>	NOT NULL	text / audio
<code>raw_text</code>	<code>text</code>	nullable	원문 텍스트 or Whisper 변환 결과
<code>audio_path</code>	<code>text</code>	nullable	Supabase Storage 경로

audio_duration	integer	nullable	녹음 길이(초)
ai_summary	text	nullable	Claude 요약 (2~3문장)
extracted	jsonb	default '{}'	AI 추출: {hidden_needs[], objections[], signals[], next_actions[]}
created_at	timestampz	default now()	기록일

4.3 contact_logs — 연락 이력

컬럼명	타입	제약	설명
id	uuid	PK	이력 ID
deal_id	uuid	FK → deals	연결된 딜
user_id	uuid	FK → auth.users	작성자
method	text	NOT NULL	전화/카카오톡/이메일/방문
initiated_by	text	NOT NULL	sales / customer
response_speed_hours	numeric	nullable	고객 응답 소요 시간
sentiment	text	nullable	positive/neutral/negative/no_response
note	text	nullable	연락 내용 메모
contacted_at	timestampz	NOT NULL	연락 일시

4.4 win_patterns — Win 공식 집계 뷰 (자동 생성)

```
CREATE OR REPLACE VIEW win_patterns AS
SELECT
  emr_vendor,
  hospital_size,
  services,
  COUNT(*) FILTER (WHERE outcome = 'win') AS win_count,
  COUNT(*) FILTER (WHERE outcome = 'loss') AS loss_count,
  COUNT(*) AS total_count,
  ROUND(
    COUNT(*) FILTER (WHERE outcome = 'win') * 100.0 / NULLIF(COUNT(*), 0), 1
  ) AS win_rate,
  ROUND(AVG(
    CASE WHEN outcome = 'win'
      THEN EXTRACT(DAY FROM (updated_at - created_at)) END
  ), 1) AS avg_days_to_win
FROM deals
WHERE outcome IS NOT NULL
  AND user_id = auth.uid()
GROUP BY emr_vendor, hospital_size, services;
```

4.5 RLS 정책

```
-- deals
ALTER TABLE deals ENABLE ROW LEVEL SECURITY;
CREATE POLICY "deals_own" ON deals
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());
```

```
-- meeting_logs, contact_logs 동일 패턴 적용
ALTER TABLE meeting_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY "logs_own" ON meeting_logs
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());

-- Storage bucket: audio-recordings
-- 정책: 본인 user_id 폴더만 읽기/쓰기 허용
-- 경로 규칙: {user_id}/{deal_id}/{timestamp}.{ext}
```

5. API 엔드포인트 스펙

Method	Path	Auth	설명
GET	/api/deals	Required	딜 목록 조회 (stage 필터, 정렬 지원)
POST	/api/deals	Required	딜 생성
GET	/api/deals/[id]	Required	딜 상세 조회 (meeting_logs 포함)
PATCH	/api/deals/[id]	Required	딜 수정 (단계 변경, 결과 기록)
DELETE	/api/deals/[id]	Required	딜 삭제
POST	/api/analyze-memo	Required	텍스트 메모 → Claude 구조화
POST	/api/transcribe	Required	음성 파일 업로드 → Whisper → 텍스트
POST	/api/coach	Required	딜 ID → AI 코치 전략 응답
POST	/api/proposal	Required	딜 ID → 제안서 초안 생성
GET	/api/win-patterns	Required	Win 공식 집계 데이터 조회
GET	/api/today-actions	Required	오늘 할 일 목록 (AI 우선순위)

5.1 POST /api/analyze-memo — 상세

Request Body

```
{
  "deal_id": "uuid",
  "text": "원장이 요즘 환자 이탈 때문에 고민 많다고 함...",
  "input_type": "text" // or "audio" (transcribe 후 전달)
}
```

Response

```
{
  "summary": "원장이 환자 이탈 문제로 고민 중, 경쟁 병원 의식",
  "extracted": {
    "hidden_needs": ["환자 이탈 방지", "경쟁 병원 대비 차별화"],
    "pain_points": ["열 병원이 앱 사용 시작"],
    "objections": [],
    "win_signals": ["ROI 관심 있음"],
    "next_actions": ["실장 미팅 준비", "경쟁 병원 사례 자료 준비"]
  },
  "context_patch": {
    "contact_preference": "전화선호",
    "decision_maker_style": "데이터중시"
  }
}
```

5.2 POST /api/transcribe — 음성 업로드 상세

처리 흐름

- 클라이언트에서 FormData 로 음성 파일 전송 (multipart/form-data)
- 서버에서 Supabase Storage 에 업로드: audio-recordings/{user_id}/{deal_id}/{ts}.{ext}

3. OpenAI Whisper API 호출: whisper-1 모델, language: ko
4. 변환된 텍스트를 /api/analyze-memo 로 내부 전달
5. meeting_logs 저장 + deals.context_snapshot 업데이트

지원 파일 형식

허용 확장자: .mp3, .mp4, .m4a, .wav, .webm, .ogg
최대 파일 크기: 25MB (Whisper API 제한)
최대 녹음 길이: 약 30분 (권장 10분 이하)
Content-Type 검증: 서버사이드에서 MIME 타입 확인 필수

비용 참고

Whisper 요금: \$0.006/분 → 10분 녹음 1회 = \$0.06. 월 50회 업로드 기준 약 \$3/월.

5.3 POST /api/coach — AI 코치 상세

Request Body

```
{
  "deal_id": "uuid"
}
```

서버 처리 로직

6. deal + meeting_logs + contact_logs 풀 조회
7. win_patterns 뷰에서 동일 세그먼트(EMR, 규모) 통계 조회
8. Claude API 호출 — 아래 프롬프트 구조 사용
9. 응답을 스트리밍으로 클라이언트 전달 (SSE)

6. AI 프롬프트 설계

모든 프롬프트는 `lib/ai/prompts.ts` 에 상수로 관리. 시스템 프롬프트는 Supabase 프롬프트 캐싱 적용하여 비용 절감.

6.1 메모 구조화 프롬프트

```
// lib/ai/prompts.ts
export const MEMO_ANALYSIS_SYSTEM = `
당신은 헬스케어 B2B 영업 전문가입니다.
영업 담당자의 미팅 메모를 분석하여 구조화된 인사이트를 추출합니다.

다음 JSON 형식으로만 응답하세요:
{
  "summary": "2~3문장 핵심 요약",
  "extracted": {
    "hidden_needs": ["고객이 말하지 않은 숨겨진 니즈"],
    "pain_points": ["명시적으로 언급한 불편/문제"],
    "objections": ["우려사항 또는 반론"],
    "win_signals": ["구매 의사 신호"],
    "next_actions": ["다음 단계 액션 아이템"]
  },
  "context_patch": {
    "contact_preference": "전화번호|카카오톡|이메일|방문번호 중 하나 (확인된 경우)",
    "decision_maker_style": "데이터중시|관계중시|빠른결정|신중함 중 하나 (확인된 경우)",
    "budget_sensitivity": "민감|보통|둔감 중 하나 (확인된 경우)"
  }
};
```

6.2 AI 코치 프롬프트

```
export const COACH_SYSTEM = `
당신은 국내 의료기관 대상 헬스케어 SaaS 영업 코치입니다.
레몬헬스케어의 서비스(레몬케어, 레몬톡톡 등)를 판매하는 영업을 지원합니다.

제공되는 정보:
- 현재 딜의 전체 맥락 (고객 프로파일, 미팅 이력, 연락 이력)
- 유사 세그먼트(EMR사, 병원 규모)의 과거 Win/Loss 통계

응답 구조 (마크다운):
## 이 딜의 Win 가능성
[S/A/B/C 등급과 근거 1~2문장]

## 지금 당장 해야 할 것
1. [가장 중요한 액션]
2. [두 번째 액션]
3. [세 번째 액션]

## 유사 Win 패턴
[동일 EMR사 또는 동일 병원 규모에서 통했던 전략]

## 주의할 것
```

```
[위험 신호 또는 놓치기 쉬운 포인트]
```

```
## 반론 대응  
[예상 반론과 대응 방법]  
`;
```

6.3 제안서 초안 프롬프트

```
export const PROPOSAL_SYSTEM = `  
당신은 레몬헬스케어 영업 제안서 작성 전문가입니다.  
고객의 맥락에 맞춘 맞춤형 제안서 초안을 작성합니다.  
  
제안서 구조:  
# [병원명] 디지털 환자 경험 개선 제안  
  
## 1. 현황 및 과제  
[고객이 언급한 pain_points 기반]  
  
## 2. 제안 솔루션  
[서비스별 기능과 도입 효과]  
  
## 3. 기대 효과  
[ROI 또는 정성적 효과]  
  
## 4. 도입 절차  
[단계별 온보딩 프로세스]  
  
## 5. 레퍼런스  
[동일 EMR사 또는 동일 규모 도입 사례 언급 - 일반적 표현 사용]  
`;
```

7. 화면 구성 상세

7.1 홈 대시보드 (/)

오늘 할 일 카드	AI가 전체 딜을 분석하여 우선순위 3가지 자동 생성. 기한 초과 딜 빨간색 강조.
파이프라인 요약	단계별 딜 수 카운트. 발굴/제안/협상/확정/계약 가로 배치.
위험 신호 목록	현재 단계 체류 7일 초과 딜 자동 감지하여 표시. 클릭 시 딜 상세로 이동.
Win 공식 미리보기	가장 최근 패턴 1~2개 요약 표시. 10개 미만이면 '데이터 누적 중' 안내.

7.2 딜 목록 (/deals)

딜 카드	병원명, 서비스, 단계 배지, 확도, 체류일(D+n), 최근 메모 요약 표시
정렬/필터	확도순 (S→C), 최근활동순, 단계별 필터 탭
새 딜 버튼	딜 생성 폼으로 이동. 현재 단계 선택 필드 포함 (어느 단계에서든 시작 가능).
딜 생성 필드	병원명*, 서비스*(다중선택), 현재단계*, 병원규모, EMR사, 담당자명/역할, 메모(선택)

7.3 딜 상세 (/deals/[id]) — 핵심 화면

화면은 4개 섹션으로 구성된다.

섹션 A — 고객 프로파일 카드

- 병원명, EMR 사, 규모, 단계, 확도 표시
- 핵심 니즈, 숨겨진 니즈, 반론, Win 신호 태그 형태로 표시
- 의사결정자 정보 (이름, 역할, 성향, 선호 연락 수단)
- 수정 버튼 클릭 시 인라인 편집 모드

섹션 B — 미팅 메모 입력

- 텍스트 입력 탭: 자유 텍스트 textarea + AI 분석하기 버튼
- 음성 업로드 탭: 드래그&드롭 또는 클릭 업로드
- 지원 형식: .mp3, .m4a, .wav, .webm
- 업로드 후 진행 상태 표시: 업로드 중 → 변환 중 → 분석 중 → 완료
- 분석 완료 후 요약 + 추출 인사이트 인라인 표시
- 미팅 로그 타임라인 (날짜 역순)

섹션 C — AI 코치

- '이 딜, 어떻게 이기나요?' 버튼 → 스트리밍 응답 표시
- 응답: Win 가능성 등급, 지금 할 것 3가지, 유사 Win 패턴, 주의 사항, 반론 대응
- 유사 딜 케이스 보기 버튼 (동일 EMR 사 + 동일 규모 딜 목록)

섹션 D — 액션 및 문서

- 단계 변경 버튼 (이전/다음 단계 이동)
- Next Action 입력 + Due Date 설정
- 결과 기록: Win 버튼, Loss 버튼 (Loss 선택 시 드랍 원인 선택 팝업)
- 제안서 초안 생성 버튼 → /deals/[id]/proposal 페이지로 이동

7.4 Win 공식 대시보드 (/win-formula)

EMR별 Win률 차트	유비케어/비트/이지케어텍 등 EMR사별 Win률 바 차트. 10개 미만 딜은 회색 처리.
병원 규모별 패턴	의원/병원/종합/상급종합 별 평균 계약 소요일 + Win률 테이블
최적 연락 패턴	contact_logs에서 집계: 평균 응답 빠른 연락 수단, 최적 연락 간격
반론 대응 라이브러리	meeting_logs extracted.objections 자주 등장 키워드 + 대응 성공 패턴
데이터 부족 상태	딜 10개 미만 시 전체 화면에 '데이터 누적 중' 안내 + 현재 딜 수 표시

8. 구현 순서 (14일 계획)

Day	파일/모듈	구현 내용	완료 기준
1~2	supabase/migrations/001_init.sql	deals, meeting_logs, contact_logs 테이블 생성. RLS 정책 적용. Storage 버킷 생성.	Supabase Studio에서 테이블 확인 + RLS 테스트 통과
3	app/(auth)/login/page.tsx	Supabase Auth 이메일 로그인. 로그인 성공 시 /로 리디렉트. 미인증 시 /login으로 강제 이동.	로그인 → 홈 리디렉트 동작 확인
4	app/(dashboard)/deals/	딜 목록 + 딜 생성 폼 구현. Supabase CRUD 연동. DealCard 컴포넌트 작성.	딜 생성 → 목록 표시 → 삭제 전체 흐름 동작
5	lib/ai/claude.ts + prompts.ts	Claude API 래퍼 함수. MEMO_ANALYSIS_SYSTEM 프롬프트 상수 정의. analyze-memo API 라우트 구현.	테스트 메모 입력 → JSON 응답 추출 확인
6	components/memo/MemoInput.tsx	딜 상세 텍스트 메모 입력 UI. 분석 버튼 → analyze-memo API 호출 → 결과 표시. context_snapshot 업데이트.	메모 입력 → AI 분석 → 화면 결과 표시 E2E 동작
7	lib/ai/whisper.ts	Whisper API 래퍼. 파일 크기/형식 검증. transcribe API 라우트 구현. Supabase Storage 업로드 로직.	m4a 파일 업로드 → 텍스트 변환 확인
8	components/memo/AudioUpload.tsx	드래그&드롭 UI. 업로드 진행 상태 표시 (4단계). 변환 완료 후 자동으로 analyze-memo 호출.	음성 업로드 → 변환 → 분석 → 화면 표시 E2E
9	app/api/coach/route.ts	딜 전체 맥락 + win_patterns 조회 → Claude COACH_SYSTEM 프롬프트. SSE 스트리밍 응답.	코치 버튼 클릭 → 스트리밍 텍스트 출력 확인
10	components/ai/CoachPanel.tsx	AI 코치 UI. 스트리밍 응답 렌더링. 마크다운 파싱 (react-markdown). 로딩 애니메이션.	코치 패널 UI 완성 + 스트리밍 표시 확인
11	app/api/proposal/route.ts	PROPOSAL_SYSTEM 프롬프트. 딜 맥락 기반 제안서 생성. /deals/[id]/proposal 페이지.	제안서 초안 생성 → 마크다운 렌더링 확인
12	app/(dashboard)/win-formula/	win_patterns 뷰 데이터 조회. EMR별 Win률 바 차트 (Recharts). 딜 10개 미만 상태 처리.	차트 표시 확인 (테스트 데이터 입력 후)
13	app/(dashboard)/page.tsx	홈 대시보드. 오늘 할 일 AI 생성 (today-actions API). 파이프라인 요약. 위험 신호 달.	홈 화면 전체 섹션 표시 확인
14	Vercel 배포 + 통합 테스트	vercel.json 설정. 환경 변수 Vercel에 등록. 전체 E2E 테스트. 주요 버그 수정.	https://xxx.vercel.app 접속 + 전체 플로우 동작

9. 주요 컴포넌트 코드 레퍼런스

9.1 타입 정의 (types/index.ts)

```
export type DealStage = 'discovery' | 'proposal' | 'negotiation' | 'confirmed'
| 'contract';
export type ProbGrade = 'S' | 'A' | 'B' | 'C';
export type Outcome = 'win' | 'loss' | 'active';

export interface ContextSnapshot {
  hidden_needs: string[];
  pain_points: string[];
  objections: string[];
  win_signals: string[];
  decision_maker_style?: string;
  contact_preference?: string;
  budget_sensitivity?: string;
  decision_log: Array<{ date: string; note: string }>;
}

export interface Deal {
  id: string;
  hospital_name: string;
  services: string[];
  stage: DealStage;
  prob_grade?: ProbGrade;
  hospital_size?: string;
  emr_vendor?: string;
  context_snapshot: ContextSnapshot;
  outcome?: Outcome;
  days_in_stage: number;
  next_action?: string;
  due_date?: string;
  created_at: string;
  updated_at: string;
}

export interface MeetingLog {
  id: string;
  deal_id: string;
  input_type: 'text' | 'audio';
  raw_text?: string;
  audio_path?: string;
  audio_duration?: number;
  ai_summary?: string;
  extracted: {
    hidden_needs: string[];
    pain_points: string[];
    objections: string[];
    win_signals: string[];
    next_actions: string[];
  };
  created_at: string;
}
```

9.2 음성 업로드 API 핵심 로직

```
// app/api/transcribe/route.ts
```

```
export async function POST(req: Request) {
  const supabase = createServerClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return new Response('Unauthorized', { status: 401 });

  const formData = await req.formData();
  const file = formData.get('file') as File;
  const dealId = formData.get('deal_id') as string;

  // 1. Supabase Storage 업로드
  const path = `${user.id}/${dealId}/${Date.now()}.${ext}`;
  await supabase.storage.from('audio-recordings').upload(path, file);

  // 2. Whisper API 호출
  const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
  const transcription = await openai.audio.transcriptions.create({
    file,
    model: 'whisper-1',
    language: 'ko',
  });

  // 3. Claude 구조화 호출 (analyze-memo 재사용)
  const analysis = await analyzeMemo(transcription.text, dealId);

  // 4. DB 저장
  await supabase.from('meeting_logs').insert({
    deal_id: dealId, user_id: user.id,
    input_type: 'audio', raw_text: transcription.text,
    audio_path: path, ai_summary: analysis.summary,
    extracted: analysis.extracted,
  });

  return Response.json({ success: true, analysis });
}
```


10. MAKE 완료 체크리스트

Day 14 기준 모든 항목이 통과되어야 MVP 완료

- Supabase 프로젝트 생성 + 스키마 마이그레이션 완료
- RLS 정책 적용 + 본인 데이터만 접근 가능 확인
- 로그인 → 홈 리디렉트 동작
- 딜 생성 → 목록 표시 → 삭제 전체 흐름
- 텍스트 메모 입력 → Claude 분석 → 결과 표시
- 음성 파일 업로드 (.m4a) → Whisper 변환 → Claude 분석 → 결과 표시
- AI 코치 버튼 → 스트리밍 응답 출력
- 제안서 초안 생성 → 마크다운 렌더링
- 단계 변경 → stage_log 업데이트 확인
- Win/Loss 결과 기록 → win_patterns 뷰 반영 확인
- Win 공식 화면 → EMR 별 차트 표시 (테스트 데이터 기준)
- 홈 화면 → 오늘 할 일 + 위험 신호 딜 표시
- Vercel 배포 완료 + HTTPS 접속 확인
- 전체 E2E 시나리오 1 회 통과: 딜 생성 → 음성 메모 → AI 코치 → 제안서